HW3 Formal Semantics

Ontology Vocabulary and SHACL Validation, Stated Mathematically

NYCU Physical AI / TAICA — TA INTERNAL, DO NOT DISTRIBUTE

Course Staff

Version 2026-fall-3.0 (matches `hw3-ontology.ttl` v3.1 and the released shapes)

This note restates, with mathematical precision, what the course materials define operationally: the vocabulary of `semantic/ontology/hw3-ontology.ttl` (§2–§3) and the meaning of the SHACL layer (§4–§6). Everything here is checkable against the released files; nothing introduces new requirements.

1 Preliminaries: graphs, predicates, literals

Definition 1.1 (Data graph)**.** *An RDF* data graph G *is a finite set of triples* (s, p, o) *over IRIs, blank nodes, and literals. We write* $\Delta(G)$ *for the set of subjects and objects occurring in* G *(the* domain *of discourse).*

Definition 1.2 (Predicate reading)**.** *For a class IRI* C *and node* x *we write*

$$C(x) \iff (x,\ \textit{rdf:type},\ C) \in G,$$

and for a property IRI p *and nodes/literals* x, y*,*

$$p(x, y) \iff (x,\ p,\ y) \in G.$$

All quantifiers in this note range over $\Delta(G)$ *of the graph being validated, and* ***only*** *over it (closed world; see §4.3). No inference is applied:* $C(x)$ *requires the* explicit *typing triple, and subclass axioms of §2 do* not *extend the extension of* C *during validation.*

Definition 1.3 (Typed literals)**.** *A literal* v *carries a datatype* $\mathrm{dt}(v)$*. Numeric comparison* $v \le c$ *is defined for numeric datatypes (xsd:double, xsd:decimal, xsd:integer, . . .) by their value spaces; for non-numeric* v *the comparison is* undefined*. The course serializers emit every float as "r"^^xsd:double; a bare Turtle decimal such as* `0.0892` *has* $\mathrm{dt} =$ *xsd:decimal — numerically comparable, but datatype-distinct (this distinction matters in Def. 4.4).*

2 The vocabulary (signature)

The signature $\Sigma = (\mathbf{C}, \mathbf{P}_o, \mathbf{P}_d)$ consists of class names, object properties, and datatype properties. Reused terms keep their original IRIs (declared as offline stubs); hw3: mints only what no standard covers.

2.1 Classes

Class	Origin	Subclass axioms	Intended extension
cora:Robot	IEEE 1872 CORA	—	the UR5 individual
soma:Joint	SOMA	—	kinematic joints
soma:RevoluteJoint	SOMA	$\sqsubseteq$ soma:Joint	the six UR5 joints
soma:JointState	SOMA	—	one angle of one joint
soma:6DPose	SOMA	$\sqsubseteq$ pos:Pose	pose nodes
hw3:KinematicComputation	minted	—	FK/IK occurrences
hw3:FKComputation	minted	$\sqsubseteq$ hw3:KinematicComputation	FK executions
hw3:IKComputation	minted	$\sqsubseteq$ hw3:KinematicComputation	IK executions
hw3:JointConfiguration	minted	—	full-arm configurations
hw3:QuaternionOrientation	minted	$\sqsubseteq$ pos:Orientation	(q_x, q_y, q_z, q_w) nodes

Subclass axioms are *modelling documentation*: they align terms to the reused standards, but by Def. 1.2 they play no role in validation. The serializers therefore always assert the most specific class expected by the shapes.

2.2 Object properties $\mathbf{P}_o$ (with signatures $p \subseteq C \times D$)

Property	Signature	Super-property
hw3:hasJoint	cora:Robot $\times$ soma:Joint	
hw3:computedForRobot	hw3:KinematicComputation $\times$ cora:Robot	
hw3:solvesForTarget	hw3:IKComputation $\times$ soma:6DPose	
hw3:hasInputJointConfiguration	hw3:FKComputation $\times$ hw3:JointConfiguration	
hw3:hasJointConfiguration	hw3:IKComputation $\times$ hw3:JointConfiguration	
hw3:hasEndEffectorPose	hw3:KinematicComputation $\times$ soma:6DPose	
hw3:hasJointState	hw3:JointConfiguration $\times$ soma:JointState	
hw3:isStateOfJoint	soma:JointState $\times$ soma:Joint	
hw3:hasPositionComponent	pos:Pose $\times$ pos:Position	
hw3:hasOrientationComponent	pos:Pose $\times$ pos:Orientation	
qudt:hasUnit	(any) $\times$ {unit:M, unit:RAD}	

2.3 Datatype properties $\mathbf{P}_d$ (with signatures $p \subseteq C \times val(d)$)

Property	Domain	Range	Read by
hw3:hasDoF	cora:Robot	xsd:integer	STRUCTURE
hw3:jointIndex	soma:Joint	xsd:integer	STRUCTURE
hw3:dh_a, hw3:dh_d, hw3:dh_alpha	soma:Joint	xsd:double	STRUCTURE
hw3:hasJointLowerLimit, hw3:hasJointUpperLimit	soma:Joint	xsd:double	Φ_{JL} (Def. 5.4)
soma:hasJointPosition	soma:JointState	xsd:double	Φ_{JL}
hw3:hasIKStatus	hw3:IKComputation	xsd:string	informational
hw3:hasResidual	hw3:KinematicComputation	xsd:double	Φ_{NC}
hw3:hasTargetDistance	hw3:IKComputation	xsd:double	Φ_{AR}
hw3:hasPoseError, hw3:hasJacobianError	hw3:FKComputation	xsd:double	Φ_{FK}
hw3:hasPositionX/Y/Z	pos:Position	xsd:double	STRUCTURE
hw3:hasQuatX/Y/Z/W	hw3:QuaternionOrientation	xsd:double	—
hw3:producedBy	(any)	xsd:string	provenance

3 Well-formed grounding graphs

The following definitions state what a correct `data.ttl` *is*; the STRUCTURE shapes of §5.2 are their operational check. Fix constants: number of joints $n = 6$; course thresholds $\theta_{\mathrm{FK}} = 0.005$, $\theta_{\mathrm{AR}} = 0.90$, $\theta_{\mathrm{NC}} = 0.02$ (metres).

Definition 3.1 (Robot specification)**.** *A node r is a* well-formed robot specification *iff* $\mathsf{cora{:}Robot}(r)$*,* $\mathsf{hw3{:}producedBy}(r,\cdot)$*,* $\mathsf{hw3{:}hasDoF}(r,6)$*, and*

$$\big|\{\, j : \mathsf{hw3{:}hasJoint}(r,j)\,\}\big| = 6,$$
$$\forall j\ \big[\mathsf{hw3{:}hasJoint}(r,j) \rightarrow \mathsf{soma{:}RevoluteJoint}(j)\big],$$

and every such j carries hw3:jointIndex*,* hw3:dh_a/d/alpha*, and both limit values, each with the exact datatype of its range column above.*

Definition 3.2 (Joint configuration)**.** *A node c is a* well-formed joint configuration *iff* $\mathsf{hw3{:}JointConfiguration}(c)$ *and*

$$\big|\{\, s : \mathsf{hw3{:}hasJointState}(c,s)\,\}\big| = 6,$$
$$\forall s\ \big[\mathsf{hw3{:}hasJointState}(c,s) \rightarrow \mathsf{soma{:}JointState}(s)$$
$$\wedge\ \exists j\ \mathsf{hw3{:}isStateOfJoint}(s,j) \wedge \exists v\ \mathsf{soma{:}hasJointPosition}(s,v)\big].$$

Definition 3.3 (Structured pose)**.** *A node π is a* well-formed pose *iff* $\mathsf{soma{:}6DPose}(\pi)$ *and* $\exists\,\rho\ [\mathsf{hw3{:}hasPositionComponent}(\pi,\rho) \wedge \mathsf{pos{:}Position}(\rho) \wedge \mathsf{hw3{:}hasPositionX/Y/Z}(\rho,\cdot)]$*, with an analogous orientation component. No pose, configuration, or numeric vector is ever encoded as a string literal (*no-JSON-strings rule*).*

Definition 3.4 (FK / IK computation records)**.** *x is a well-formed* FK record *iff* $\mathsf{hw3{:}FKComputation}(x)$ *with exactly one input configuration (Def. 3.2), exactly one end-effector pose (Def. 3.3), and* xsd:double*-typed* hw3:hasPoseError*,* hw3:hasJacobianError*, plus provenance. x is a well-formed* IK record *iff* $\mathsf{hw3{:}IKComputation}(x)$ *with one target link, one output configuration, a status string, and* xsd:double*-typed* hw3:hasResidual*,* hw3:hasTargetDistance*, plus provenance.*

4 SHACL validation, formally

4.1 The validation function

Definition 4.1 (Validation)**.** *Given a shapes graph $\mathcal{S}$ and data graph G, validation computes a finite set of* results

$$\mathrm{Report}(\mathcal{S},G) \;=\; \{\,(x,\ m,\ p,\ v)\ :$$
$$\textit{focus node } x \textit{ violates a constraint with message } m \textit{ on path } p \textit{ at value } v\,\},$$

and $\mathrm{conforms}(\mathcal{S},G) \Leftrightarrow \mathrm{Report}(\mathcal{S},G) = \emptyset$*. For a shape with* sh:targetClass *C, the focus nodes are $\{x : C(x)\}$ (explicit typing only, Def. 1.2); for each* sh:property [sh:path *p* ; …] *the* value nodes *of x are $\{v : p(x,v)\}$.* ***Each*** *failing value node contributes one result — results are per-violation, not per-shape.*

4.2 Constraint components used in this course

Definition 4.2 (Value range, sh:maxInclusive c)**.**

$$\mathrm{ok}(x) \iff \forall v\,\big[p(x,v) \rightarrow v \le c\big], \qquad \mathrm{viol}(x,v) \iff p(x,v) \wedge \neg(v \le c).$$

Note $\neg(v \le c)$, not $v > c$: an incomparable v (e.g. a string) also violates. The exclusive variant sh:maxExclusive *replaces $v \le c$ by $v < c$; the S3 boundary records ($v = c$ exactly) separate the two.*

Definition 4.3 (Cardinality, sh:minCount m / sh:maxCount k)**.**

$$\mathrm{ok}(x) \iff m \le \big|\{\, v : p(x,v) \,\}\big| \le k.$$

Definition 4.4 (Datatype, sh:datatype d)**.**

$$\mathrm{ok}(x) \iff \forall v \big[p(x,v) \to v \text{ is a literal} \wedge \mathrm{dt}(v) = d \big].$$

This is datatype identity*:* $\mathrm{dt}(0.0892) = \mathsf{xsd{:}decimal} \neq \mathsf{xsd{:}double}$*, hence the classic* STRUCTURE *deduction, even though the same value passes every numeric comparison of Def. 4.2.*

Definition 4.5 (Class, sh:class C)**.** $\mathrm{ok}(x) \Leftrightarrow \forall y\,[p(x,y) \to C(y)]$*, with* $C(y)$ *explicit as always.*

Definition 4.6 (SPARQL constraint, sh:sparql)**.** *A query* $Q(this)$ *is evaluated once per focus node* x *with* $this \mapsto x$*;* ***every solution mapping*** μ *of* Q *yields one violation* $(x, m, -, \mu)$*. Thus* Q*'s body must be the* negation *of the conformance condition.*

4.3 Closed world

Convention 4.7 (Negation as failure)**.** *Validation treats* G *as a complete description: a triple not in* G *is false. Contrast OWL's open-world reading, under which absence is unknown — and under which numeric comparison is not even expressible. Hence the course invariant:* the ontology carries no thresholds and no cardinalities; all checking is SHACL *(TA guide §4.4).*

Lemma 4.8 (Vacuous conformance)**.** *If* x *has no* p*-value, then* x *satisfies every value-range, datatype, and class constraint on* p *(the universal quantifier is vacuous), and violates a* sh:minCount m *constraint with* $m \ge 1$*. Presence and correctness are therefore enforced by* different *shapes, and both are needed.*

5 The course shapes as formulas

5.1 Problem shapes (student `shapes.ttl`)

Definition 5.1 (FK accuracy, worked example)**.**

$$\Phi_{\mathrm{FK}}: \quad \forall x\, \forall v \ \big[\mathsf{hw3{:}FKComputation}(x) \wedge \mathsf{hw3{:}hasPoseError}(x,v) \ \to \ v \le \theta_{\mathrm{FK}} \big]$$

with flag FK_INACCURATE *on each violation.*

Definition 5.2 (Arm out of range, TODO 1)**.**

$$\Phi_{\mathrm{AR}}: \quad \forall x\, \forall v \ \big[\mathsf{hw3{:}IKComputation}(x) \wedge \mathsf{hw3{:}hasTargetDistance}(x,v) \ \to \ v \le \theta_{\mathrm{AR}} \big]$$
flag ARM_OUT_OF_RANGE*.*

Definition 5.3 (No convergence, TODO 2)**.**

$$\Phi_{\mathrm{NC}}: \quad \forall x\, \forall v \ \big[\mathsf{hw3{:}IKComputation}(x) \wedge \mathsf{hw3{:}hasResidual}(x,v) \ \to \ v \le \theta_{\mathrm{NC}} \big]$$
flag NO_CONVERGENCE*.*

Definition 5.4 (Joint limit, TODO 3, SHACL-SPARQL)**.**

$$\Phi_{\mathrm{JL}}: \quad \forall s\, \forall v\, \forall j\, \forall \ell\, \forall u \ \Big[\mathsf{soma{:}JointState}(s) \wedge \mathsf{soma{:}hasJointPosition}(s,v) \wedge \mathsf{hw3{:}isStateOfJoint}(s,j)$$
$$\wedge\ \mathsf{hw3{:}hasJointLowerLimit}(j,\ell) \wedge \mathsf{hw3{:}hasJointUpperLimit}(j,u) \ \to \ \ell \le v \le u \Big]$$
flag JOINT_LIMIT_VIOLATION*.*

SHACL Core relates a focus node only to its own values; Φ_{JL} joins values across two nodes (s and j), so it is realised as a SPARQL constraint whose `FILTER` *is the negation of the consequent:* `FILTER (?v < ?lo || ?v > ?hi)` *(Def. 4.6). Limits are inclusive; a non-strict filter (<=, >=) implements $\ell < v < u$ and falsely flags the boundary record* `jl_case_02` *($v = u$ exactly). By closed world (Lemma 4.8), a joint state whose joint declares no limits yields no query solution and conforms vacuously; on* `data.ttl` *Def. 3.1 guarantees limits exist.*

5.2 Structure shapes (TA `ta-shapes-full.ttl`)

The STRUCTURE shapes are exactly the operational encodings of Definitions 3.1–3.4, assembled from the components of §4: e.g. "a robot has exactly six revolute joints" is sh:minCount 6 ∧ sh:maxCount 6 ∧ sh:class soma:RevoluteJoint on path hw3:hasJoint; every numeric requirement of Def. 3.4 is sh:minCount 1 ∧ sh:datatype xsd:double. The TA suite additionally contains the problem shapes $\Phi_{\mathrm{AR}}, \Phi_{\mathrm{NC}}$ (for S2) and its own copy of Φ_{JL}.

6 Flags, answer key, and scoring semantics

Convention 6.1 (Flag extraction)**.** *For a result (x, m, p, v), the machine-readable flag is the prefix of m before the first colon. For a record identifier ι (e.g.* `ik_case_09`*),*

$$\mathrm{flags}(\iota) \;=\; \{\, \mathrm{prefix}(m) \;:\; (x, m, p, v) \in \mathrm{Report}(\mathcal{S}_{\mathrm{student}},\ G_{\mathrm{TA}}),\ \iota \text{ is a substring of } x \,\}.$$

Identifiers keep a fixed digit width so that the substring relation is a bijection onto records.

Definition 6.2 (S3 — answer-key agreement)**.** *Let $K : \mathcal{I} \to 2^F$ be the answer key over record set $\mathcal{I}$ ($|\mathcal{I}| = 30$), F the four flags, and let $\mathcal{I}^+ = \{\iota : K(\iota) \neq \emptyset\}$ ($|\mathcal{I}^+| = 14$), $\mathcal{I}^0 = \mathcal{I} \setminus \mathcal{I}^+$ ($|\mathcal{I}^0| = 16$). A record is* correct *iff* $\mathrm{flags}(\iota) \overset{set}{=} K(\iota)$ *(set equality: no false positives, no false negatives). Then*

$$S_3 \;=\; 8 \cdot \frac{|\{\iota \in \mathcal{I}^+ : \mathrm{flags}(\iota) \overset{set}{=} K(\iota)\}|}{|\mathcal{I}^+|} \;+\; 2 \cdot \frac{|\{\iota \in \mathcal{I}^0 : \mathrm{flags}(\iota) \overset{set}{=} K(\iota)\}|}{|\mathcal{I}^0|} \;\in\; [0, 10].$$

Definition 6.3 (S1, S2, and the Task 1 score)**.** *With V_{STRUCT} the set of STRUCTURE-prefixed results of the TA suite on the student's* `data.ttl`*:*

$$S_1 = \max\bigl(0,\ 12 - 2\,|V_{\mathrm{STRUCT}}|\bigr),$$
$$S_2 = 2\,[\,\mathit{AR@mid}\,] + 2\,[\,\mathit{AR@far}\,] + 2\,[\,\mathit{near\ \ clean}\,] + 2\,[\,\mathit{JL\ \ absent}\,],$$

where $[\cdot]$ is the Iverson bracket, and the Task 1 score is $S_1 + S_2 + S_3 \in [0, 30]$ with pass gate (exit code 0) at ≥ 29.9. Note conforms *itself is* not *the gate: a correct solution's own data legitimately contains four violations (*mid/far *genuinely lie beyond θ_{AR}).*

Remark 6.4 (Tasks 2–4 feedback bridge)**.** *The diagnosis of* `diagnose.py` *evaluates* $\mathrm{Report}(\mathcal{S}_{\mathrm{student}},\ G_{\mathrm{exec}})$ *where G_{exec} is built, at run time, by the student's own grounding functions from the numeric execution records of Tasks 2–4, and prints* $\mathrm{flags}(\iota)$ *per record. Formally it is the same validation function (Def. 4.1) applied to a freshly grounded graph — no separate mechanism.*

Remark 6.5 (Relation to description logic)**.** *Each Φ of §5 lies in the two-variable-with-counting fragment of FOL except Φ_{JL} (five variables, a genuine join), which is exactly why it exceeds SHACL Core and OWL alike and requires the SPARQL escape hatch. The ontology's subclass axioms are ordinary $\mathcal{ALC}$ TBox inclusions; they document intent but, under closed-world validation without inference, carry no operational weight.*